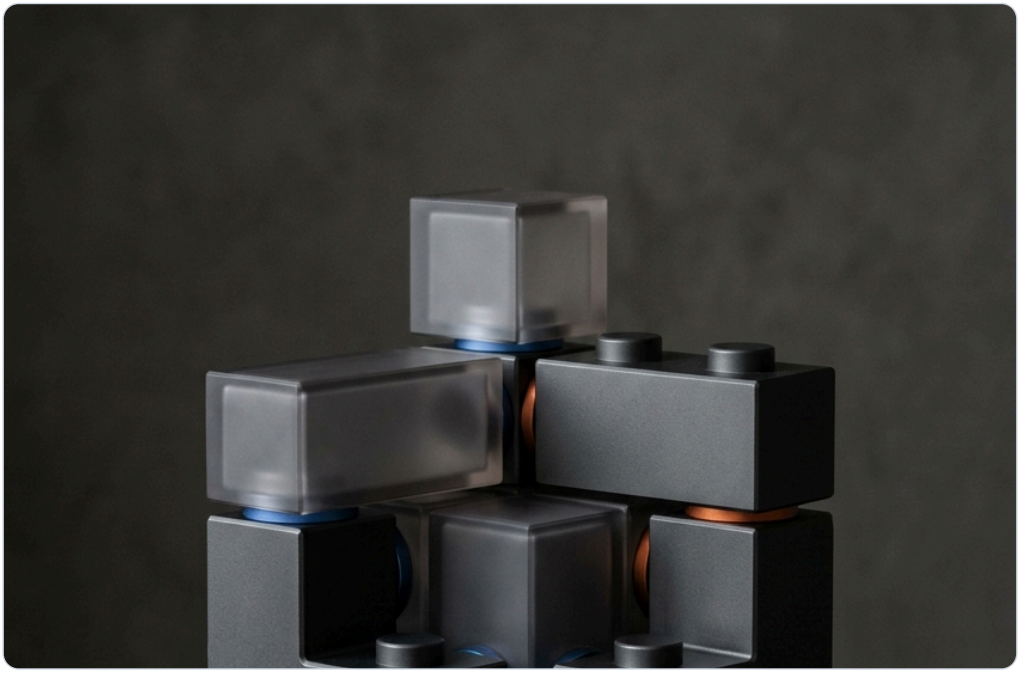
GUIA DIDÁTICO · BEHAVIOR-DRIVEN DEVELOPMENT

LegoLoop

Verbos e Granularidade no LegoLoop

Guia Didático BDD da Tríade de Triggers e do Encadeamento de Verbos



Heverton & Ecossistema LegoLoop

07/11/2026 · LegoLoop Foundation

SOBRE ESTA OBRA

Este guia ensina a operar o LegoLoop por meio de seus verbos canônicos e dos cinco níveis de granularidade que organizam qualquer fluxo: do Mega Pipeline às Nano-Etapas.

A obra percorre três camadas. No nível Macro, apresenta a filosofia de blocos componíveis governados por um grafo acíclico dirigido e a Tríade de Triggers (Terminal, Chat do Agente e LegoLoop Studio). No nível Meso, documenta as cinco trilhas canônicas de encadeamento de verbos. No nível Micro, especifica sete cenários BDD em que o mesmo comportamento é acionado pelas três vias, com resultado idêntico e auditável.

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 07/11/2026 · LegoLoop Foundation

Sumário

Como ler este guia **2**
Público e pré-requisitos 2
Estrutura da obra 2
Convenção BDD 2
PARTE I — Macro: Filosofia e Triggers **3**
1. A Filosofia Lego 3
2. Os Cinco Níveis de Granularidade 3
3. A Tríade de Triggers 3
PARTE II — Meso: Encadeamento **5**
4. Verbos como Elos de uma Corrente 5
5. As Cinco Trilhas Canônicas 5
6. Tabela-Resumo dos Encadeamentos 7
PARTE III — Micro: Cenários BDD **8**
7. Como Ler os Cenários 8
8. Cenário 1: Fabricar um Pipeline do Zero 8
9. Cenário 2: Inspecionar o Inventário com `listar` 9
10. Cenário 3: Avaliar a Saúde das Peças com `matriz` 10
11. Cenário 4: Compor um Sub-Loop em Pipeline Existente 11
12. Cenário 5: Refinar Cirurgicamente uma Etapa 12
13. Cenário 6: Remover com Segurança e Rollback 13
14. Cenário 7: Personalizar Micro e Nano-Etapas no `ROUTE.json` 13
PARTE IV — Referência Rápida **15**
15. A Tríade de Triggers por Operação 15
16. Síntese 15

Como ler este guia

Público e pré-requisitos

O guia se destina a desenvolvedores, engenheiros de IA e arquitetos de software que operam o ecossistema LegoLoop e AIDD. Pressupõe familiaridade com linha de comando e com a noção de pipeline; nenhum conhecimento prévio do motor interno é exigido.

Estrutura da obra

Parte	Nível	Conteúdo
I	Macro	Filosofia Lego, níveis de granularidade e Tríade de Triggers
II	Meso	Matriz de encadeamento e as cinco trilhas canônicas
III	Micro	Sete cenários BDD multi-trigger
IV	Referência	Tabela de consulta rápida e síntese final

Convenção BDD

Cada cenário segue a forma **Dado-Quando-Então**. O bloco *Dado* fixa o estado inicial, o bloco *Quando* descreve a ação nas três vias de acionamento, e o bloco *Então* declara o resultado verificável. O comportamento contratado não depende da via escolhida.

PARTE I — Macro: Filosofia e Triggers

1. A Filosofia Lego

O LegoLoop transforma fluxos de software e de inteligência artificial em **blocos modulares componíveis**, governados por um grafo acíclico dirigido (DAG).

Nenhum fluxo é código estático ou monolítico. Qualquer processo pode ser montado, desmontado, inspecionado, refinado ou recombinado sem efeitos colaterais nos demais blocos.

2. Os Cinco Níveis de Granularidade



Os cinco níveis de granularidade, do Mega Pipeline às Nano-Etapas

Nível	Escopo	O que se controla	Exemplo real
1. Mega Pipeline	Ciclo de vida do produto	Orquestração e paralelismo de épicos	Mega Pipeline ponta a ponta
2. Sub-Loops	Módulos reutilizáveis	Injeção hierárquica, saída isolada	Loop de Auditoria 4F
3. Etapas (Nós)	Contratos de I/O	Troca de peças, dependências	`e01` -> `e02`
4. Micro-Etapas	Execução cognitiva	Modelo de IA, contexto, MCPs	Redator de testes isolado
5. Nano-Etapas	Invariantes de código	Asserções, cobertura, gates `SG-*`	Função Go com até 35 linhas

3. A Tríade de Triggers

O LegoLoop não se limita à linha de comando. Todo verbo do ecossistema aceita **três triggers equivalentes de entrada**, e todos despacham para o mesmo motor canônico.



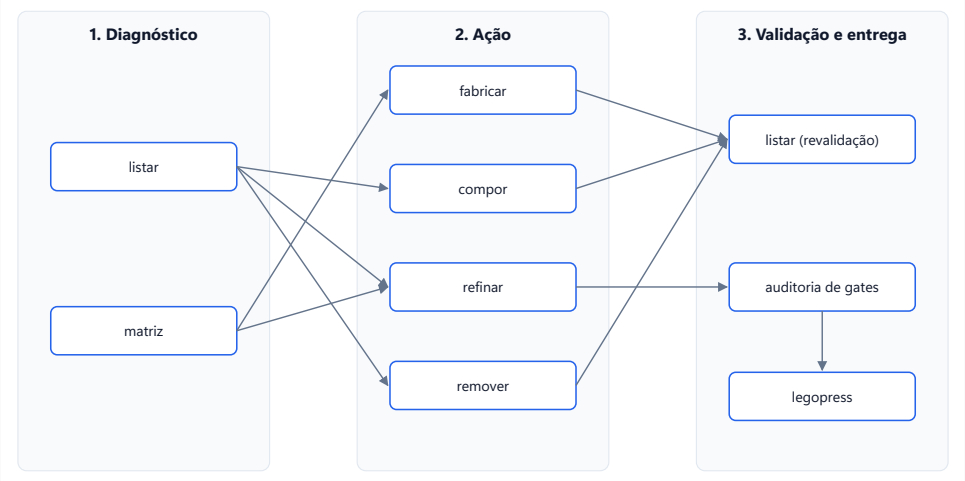
A Tríade de Triggers alimentando o motor canônico

1. **Terminal (CLI):** comando direto `legoloop <verbo> [flags]`. Indicado para scripts, CI/CD e automação sem interface.
2. **Chat do Agente (Harness):** linguagem natural no editor (Claude Code, Cursor, Windsurf, MIMO, Antigravity) por meio das skills canônicas ou slash commands, como `/legoloop-fabricar` e `/legoloop-listar`. O agente interpreta o pedido e despacha a ação de forma estéril.
3. **LegoLoop Studio (Interface Web):** painel visual aberto com `legoloop studio`. Indicado para inspeção de grafos DAG em tempo real, cartões, modais assistidos e drag-and-drop.

PARTE II — Meso: Encadeamento

4. Verbos como Elos de uma Corrente

Os verbos do LegoLoop não foram projetados para rodar isolados. Cada um é elo de uma corrente lógica: os diagnósticos alimentam ações cirúrgicas, e as ações desembocam em revalidação e entrega.



Fluxo entre diagnóstico, ação e validação

Origem	Destino	Finalidade da ligação
`listar`	`refinar`	Inspecionar os nós antes da troca
`listar`	`compor`	Identificar sub-loops reutilizáveis
`listar`	`remover`	Auditar resíduos e vínculos
`matriz`	`refinar`	Detectar peças defasadas
`matriz`	`fabricar`	Validar a aptidão das peças
`fabricar` / `compor` / `remover`	`listar`	Revalidar grafo, acoplamento e limpeza
`refinar`	Auditoria de gates	Validar o score após a troca
Auditoria de gates	`legopress`	Gerar a documentação final

5. As Cinco Trilhas Canônicas

As trilhas abaixo esgotam as combinações lógicas da operação diária.

Trilha 1: Criação e Publicação

Objetivo: criar um processo com governança completa sem duplicar trabalho.

Encadeamento: `listar` → `fabricar` → `matriz`

1. `listar`**:** verifica se já existe processo similar registrado no cofre.
2. `fabricar`**:** gera os quatro artefatos contratuais (`INTENT`, `SPEC`, `PLAN`, `ROUTE`).
3. `matriz`**:** confere se as peças selecionadas pelo motor têm boa reputação (Tier Ouro ou Prata).

Trilha 2: Diagnóstico e Refinamento Cirúrgico

Objetivo: localizar etapas de baixo desempenho e substituir peças defasadas.

Encadeamento: `listar` → `matriz` → `refinar` → `matriz`

1. `listar`**:** localiza os identificadores exatos das etapas (`e01`, `e02`).
2. `matriz`**:** filtra peças Bronze ou defasadas e mapeia substitutas Tier Ouro (≥ 0.85).
3. `refinar`**:** troca atômica da peça na etapa, com backup em `.legoloop-bak` e bump minor de versão.
4. `matriz`**:** revalida o processo e confirma que a saúde geral do cofre subiu.

Trilha 3: Composição Hierárquica e Reuso

Objetivo: reaproveitar um processo existente como sub-etapa de outro maior.

Encadeamento: `listar` → `compor` → `listar`

1. `listar`**:** seleciona o processo anfitrião (pai) e o sub-loop a ser acoplado (filho).
2. `compor`**:** conecta o sub-loop em posição específica (`legoloop compor <pai> <filho> --pos N`).
3. `listar`**:** valida a árvore hierárquica e os links do DAG na saída JSON.

Trilha 4: Ciclo de Vida e Descomissionamento

Objetivo: desativar processos depreciados, limpando symlinks de todos os editores sem quebra.

Encadeamento: `matriz` → `listar` → `remover --dry-run` → `remover --force`

1. `matriz`**:** identifica processos com peças órfãs ou critérios descontinuados.
2. `listar`**:** analisa o raio de impacto (harnesses e editores vinculados).
3. `remover --dry-run`**:** simula a exclusão para confirmar que nenhum artefato crítico será afetado.
4. `remover --force`**:** executa o descarte com limpeza de symlinks e histórico de rollback preservado.

Trilha 5: Governança Editorial e Consolidação Didática

Objetivo: transformar especificações, planos e rotas refinadas em livro técnico ou manual em PDF.

Encadeamento: `listar` → `matriz` → `legopress`

1. `listar` e `matriz`: auditam a estabilidade e a integridade dos nós do processo.
2. `legopress`: compila diffs de código, especificações BDD e diagramas em documento Typst/PDF canônico.

6. Tabela-Resumo dos Encadeamentos

Trilha	Sequência de verbos	Gatilho típico	Resultado entregue
1. Criação	listar → fabricar → matriz	Nova demanda	INTENT, SPEC, PLAN e ROUTE
2. Refinamento	listar → matriz → refinar → matriz	Peça lenta ou defasada	Etapa trocada, bump minor
3. Composição	listar → compor → listar	Reuso de fluxo	Sub-loop isolado no DAG
4. Descomissionamento	matriz → listar → remover	Processo legado	Symlinks limpos, sem órfãos
5. Fechamento editorial	matriz → legopress	Fim de ciclo	Manual e PDF canônico

PARTE III — Micro: Cenários BDD

7. Como Ler os Cenários

Cada cenário demonstra **o mesmo comportamento esperado** acionado pelas três vias. A seção *Quando* lista Terminal, Chat do Agente e Studio; as seções *Então* e *E* valem igualmente para as três.

8. Cenário 1: Fabricar um Pipeline do Zero

> ***Objetivo:*** *o sistema cria automaticamente toda a estrutura de governança para uma nova necessidade de automação.*

Dado que o desenvolvedor precisa de um pipeline para auditar a segurança de contratos inteligentes em Solidity,

Quando aciona a fabricação:

Via Terminal (CLI):

```
legoloop fabricar "auditoria-smart-contracts" \
  --goal "Auditar vulnerabilidades reentrancy e overflow em Solidity" \
  --category build --yes
```

Via Chat do Agente:

```
/legoloop-fabricar
Crie um novo processo chamado 'auditoria-smart-contracts' focado em auditar
vulnerabilidades de reentrancy e overflow em Solidity, na categoria build.
```

Via LegoLoop Studio:

1. Na aba **Processos**, clica em **+ Novo Processo**.
2. No modal, preenche **Nome** `auditoria-smart-contracts`, **Categoria** `build` e **Objetivo**.
3. Clica em **Fabricar Processo**.

Então o motor hermético produz quatro artefatos canônicos, nomeados no padrão `YYYY-MM-DD_<TIPO>_AUDITORIA_SMART_CONTRACTS`:

Artefato	Pasta	Extensão	Conteúdo
INTENT	`docs/intents/`	`.md`	Problema e motivação
SPEC	`docs/specs/`	`.md`	Critérios de aceitação binários
PLAN	`docs/plans/`	`.md`	Tickets em fatias verticais
ROUTE	`docs/routes/`	`.json`	DAG formal executável

E o pipeline fica pronto para execução sem código boilerplate manual.

9. Cenário 2: Inspecionar o Inventário com `listar`

> ***Objetivo:*** *saber quais processos existem no cofre, suas versões e a quais harnesses estão vinculados.*

Dado que existem vários loops criados na máquina,

Quando o usuário solicita a listagem:

Via Terminal (CLI):

```
legoloop listar
legoloop listar --json   # saída estruturada para pipelines
```

Via Chat do Agente:

```
/legoloop-listar
Liste todos os processos ativos no cofre do LegoLoop e verifique se há
harnesses descalibrados ou vínculos órfãos.
```

Via LegoLoop Studio:

1. Acessa a seção **Processos** no menu lateral.
2. Visualiza a grade de cartões com selo de versão (`@1.0.0`), categoria e estado de cada harness vinculado.

Então o sistema exibe nome e versão do processo, categoria (`build`, `reverse`, `web`, `mobile`), harnesses vinculados com estado dos links simbólicos e eventuais vínculos órfãos ou advertências de sincronização.

E o desenvolvedor obtém visibilidade completa do ecossistema em qualquer uma das vias.

10. Cenário 3: Avaliar a Saúde das Peças com `matriz`

> ***Objetivo:*** *garantir que o pipeline usa apenas peças modernas e de alta reputação antes de uma entrega crítica.*

Dado que o desenvolvedor quer auditar a integridade das peças do ecossistema,

Quando consulta a matriz:

Via Terminal (CLI):

```
legoloop matriz --tier ouro
```

Via Chat do Agente:

```
/legoloop-matriz
Consulte a matriz de peças do LegoLoop e mostre apenas as ferramentas
de Tier Ouro com score >= 0.85 utilizáveis no pipeline.
```

Via LegoLoop Studio:

1. Clica em **Matriz de Peças** na navegação principal.
2. Seleciona o chip **Padrão Ouro** no filtro de topo.
3. Inspeciona a tabela com scores de reputação, cobertura de testes de mutação e conformidade com zero stubs.

Então o sistema exibe apenas componentes certificados: score ≥ 0.85, sem stubs e com testes herméticos.

E sinaliza as peças Bronze ou defasadas, recomendando substituição cirúrgica.

11. Cenário 4: Compor um Sub-Loop em Pipeline Existente

> ***Objetivo:*** *anexar um sub-loop de homologação visual já existente a um pipeline de construção.*

Dado que existem o pipeline mestre `pipeline-frontend` e o sub-loop `aidd-homologue@1.0.0`,

Quando o usuário conecta o sub-loop após a etapa de build `e02`:

Via Terminal (CLI):

```
legoloop compor pipeline-frontend aidd-homologue@1.0.0 --depende e02 --yes
```

Via Chat do Agente:

```
/legoloop-compor
Componha o sub-loop 'aidd-homologue@1.0.0' dentro do 'pipeline-frontend',
fazendo-o depender da etapa 'e02'.
```

Via LegoLoop Studio:

1. Abre `pipeline-frontend` no visualizador de DAG.
2. Clica em **+ Adicionar Sub-Loop** ou arrasta o bloco `aidd-homologue` da gaveta lateral.
3. Traça a aresta de `e02` até o nó do sub-loop.
4. Clica em **Salvar Composição**.

Então o LegoLoop:

1. Verifica se a junção criaria ciclos proibidos (`ErrCiclo`).
2. Isola a saída do sub-loop em `out/aidd-homologue/`.
3. Atualiza atomicamente `.processo.json` e `.dag.json` do pipeline pai.

E o sub-loop passa a executar como caixa-preta confiável dentro do macro-fluxo.

12. Cenário 5: Refinar Cirurgicamente uma Etapa

> ***Objetivo:*** *trocar uma etapa lenta ou com ferramenta antiga sem recriar o pipeline.*

Dado que a etapa `e03_analise_estatica` usa uma ferramenta obsoleta,

Quando o usuário solicita a substituição atômica:

Via Terminal (CLI):

```
legoloop refinar "auditoria-smart-contracts" \
  --trocar e03_analise_estatica --por "slither-analyzer@2.0.0" --yes
```

Via Chat do Agente:

```
/legoloop-refinar
No processo 'auditoria-smart-contracts', refine a etapa 'e03_analise_estatica'
substituindo-a pela peça 'slither-analyzer@2.0.0'.
```

Via LegoLoop Studio:

1. Clica com o botão direito sobre o nó `e03_analise_estatica`.
2. Seleciona **Refinar / Trocar Peça** no menu de contexto.
3. Escolhe `slither-analyzer@2.0.0` no catálogo de peças aptas sugerido pela matriz.
4. Confirma no modal de validação de contratos.

Então o LegoLoop:

1. Valida a compatibilidade dos contratos de entrada e saída com os nós adjacentes.
2. Substitui o nó no DAG preservando todas as conexões.
3. Incrementa a versão menor do processo, de `@1.0.0` para `@1.1.0`.

E nenhuma outra etapa do fluxo sofre alteração ou efeito colateral.

13. Cenário 6: Remover com Segurança e Rollback

> ***Objetivo:*** *excluir um processo sem deixar resíduos nos editores nem quebrar links.*

Dado que o processo `experimento-legado` deve ser desativado,

Quando o usuário aciona a remoção segura:

Via Terminal (CLI):

```
legoloop remover "experimento-legado" --dry-run   # 1. simulação
legoloop remover "experimento-legado" --force     # 2. execução
```

Via Chat do Agente:

```
/legoloop-remover
Remova o processo 'experimento-legado' com segurança, executando a simulação
primeiro e limpando os links simbólicos de todos os harnesses.
```

Via LegoLoop Studio:

1. No cartão de `experimento-legado`, abre o menu de contexto e seleciona **Remover Processo**.
2. O diálogo de **Impacto e Dry-Run** lista arquivos e harnesses afetados.
3. O usuário digita o nome do processo e clica em **Confirmar Remoção**.

Então o sistema exibe o relatório de desvinculação, limpa os symlinks nos harnesses, arquiva os dados e gera cópia de segurança para restauração imediata.

14. Cenário 7: Personalizar Micro e Nano-Etapas no `ROUTE.json`

> ***Objetivo:*** *controlar custo cognitivo e velocidade ajustando modelo de IA e limites por bloco.*

Dado que o arquivo `docs/routes/YYYY-MM-DD_ROUTE_EXEMPLO.json` define a execução por nós,

Quando o usuário ajusta as propriedades declarativas de um nó:

Via Terminal ou editor de código:

```
{
  "id": "ticket-01",
  "titulo": "Gerar Casos de Teste Unitários",
  "modelo_cognitivo": "gemini-flash",
  "contexto_esteril": true,
  "mcps_permitidos": ["codebase-memory-mcp"],
  "stop_gates": ["SG-01-REVISAO-HUMANA"]
}
```

Via Chat do Agente:

```
No arquivo de rota mais recente, configure o nó 'ticket-01' para usar o modelo
'gemini-flash', contexto estéril e apenas a ferramenta 'codebase-memory-mcp'.
```

Via LegoLoop Studio:

1. Clica no nó `ticket-01` para abrir o painel **Propriedades da Etapa**.
2. Seleciona `gemini-flash` no dropdown de modelo.
3. Ativa o switch **Contexto Estéril**.
4. Marca apenas o MCP `codebase-memory-mcp` e ativa o portão `SG-01-REVISAO-HUMANA`.
5. Clica em **Aplicar Alterações**.

Então, na execução pela Orchestrix:

1. O nó roda com o modelo econômico `gemini-flash`.
2. Carrega apenas `codebase-memory-mcp`, sem poluir a janela de contexto.
3. Pausa obrigatoriamente no portão humano `SG-01-REVISAO-HUMANA` antes de gravar em disco.

PARTE IV — Referência Rápida

15. A Tríade de Triggers por Operação

Na coluna Terminal, todo comando é prefixado por `legoloop`; na coluna Chat, toda skill é prefixada por `/legoloop-`.

Ação desejada	Nível	Terminal	Chat	LegoLoop Studio
Criar processo	Macro	`fabricar "<nome>"`	`fabricar`	Botão + **Novo Processo**
Consultar inventário	Macro	`listar [--json]`	`listar`	Aba **Processos**
Auditar reputação	Meso	`matriz [--tier ouro]`	`matriz`	Aba **Matriz**
Encaixar sub-loop	Meso	`compor <pai> <filho>`	`compor`	Arrastar bloco e traçar aresta
Substituir etapa	Micro	`refinar "<alvo>"`	`refinar`	Botão direito no nó, **Refinar**
Remover com rollback	Macro	`remover "<nome>"`	`remover`	Menu do cartão, **Remover**
Ajustar modelo e contexto	Micro/ Nano	Edição do `ROUTE.json`	Pedido livre	Painel de propriedades do nó
Compilar livro em PDF	Macro	`legopress "<alvo>"`	`legopress`	Botão **Exportar Livro Didático**

16. Síntese

Com a Tríade de Triggers integrada ao padrão BDD e os encadeamentos canônicos mapeados, o LegoLoop deixa de ser apenas uma CLI:

- Em automação headless ou CI/CD, use o **Terminal**.
- Em programação pareada com um modelo de linguagem no editor, use o **Chat do Agente**.
- Para visão panorâmica e edição visual, use o **LegoLoop Studio**.

Em qualquer dos três caminhos, o comportamento contratado pelo BDD (**Dado-Quando-Então**) é idêntico, determinístico e auditável.